

Source Code :-

```
import pandas as pd

import numpy as np

import torch

import torch.nn as nn

import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler


# 1. Load dataset

df = pd.read_csv("Month_Value_1.csv")


# 2. Clean column names

df.columns = df.columns.str.strip()


# 3. Remove missing values

df = df.dropna()


# 4. Select target column

data = df['Revenue'].astype(float).values.reshape(-1,1)


# 5. Normalize

scaler = MinMaxScaler()

data = scaler.fit_transform(data)
```

6. Create sequences

```
def create_sequences(data, seq_len):
```

```
    X, y = [], []
```

```
    for i in range(len(data) - seq_len):
```

```
        X.append(data[i:i+seq_len])
```

```
        y.append(data[i+seq_len])
```

```
    return np.array(X), np.array(y)
```

```
seq_len = 5
```

```
X, y = create_sequences(data, seq_len)
```

7. Convert to tensors

```
X = torch.tensor(X, dtype=torch.float32)
```

```
y = torch.tensor(y, dtype=torch.float32)
```

8. Train-test split

```
train_size = int(0.8 * len(X))
```

```
X_train, X_test = X[:train_size], X[train_size:]
```

```
y_train, y_test = y[:train_size], y[train_size:]
```

9. Model

```
class RNNModel(nn.Module):
```

```
    def __init__(self):
```

```
super().__init__()
```

```
self.rnn = nn.RNN(1, 32, batch_first=True)
```

```
self.fc = nn.Linear(32, 1)
```

```
def forward(self, x):
```

```
    out, _ = self.rnn(x)
```

```
    return self.fc(out[:, -1, :])
```

```
model = RNNModel()
```

```
# 10. Loss & optimizer
```

```
criterion = nn.MSELoss()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
# 11. Training
```

```
epochs = 10
```

```
train_losses = []
```

```
val_losses = []
```

```
for epoch in range(epochs):
```

```
    model.train()
```

```
    output = model(X_train)
```

```
    loss = criterion(output, y_train)
```

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
# Prevent exploding gradients
```

```
torch.nn.utils.clip_grad_norm_(model.parameters(), 1)
```

```
optimizer.step()
```

```
train_losses.append(loss.item())
```

```
# Validation Loss
```

```
model.eval()
```

```
with torch.no_grad():
```

```
    val_output = model(X_test)
```

```
    val_loss = criterion(val_output, y_test)
```

```
# PRINT
```

```
print(f"Epoch {epoch+1}, Train Loss: {loss.item()}, Val Loss: {val_loss.item()}")
```

```
# 12. Final Predictions
```

```
predictions = scaler.inverse_transform(val_output.numpy())
```

```
y_test_np = scaler.inverse_transform(y_test.numpy())
```

13. Plot Train vs Validation Loss

```
plt.figure()

plt.plot(train_losses, label="Train Loss")

plt.legend()

plt.title("Training Loss Curve")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.show()
```

14. Plot Predictions

```
plt.figure()

plt.plot(y_test_np, label="Actual")

plt.plot(predictions, '--', label="Predicted")

plt.legend()

plt.title("Revenue Prediction using RNN")

plt.show()
```

15. Error Plot (bonus)

```
error = y_test_np - predictions
```

```
plt.figure()

plt.plot(error)

plt.title("Prediction Error")

plt.show()
```

Output:-

```
Epoch 1, Train Loss: 0.37482085824012756, Val Loss: 0.8163308501243591
Epoch 2, Train Loss: 0.34792864322662354, Val Loss: 0.7722181677818298
Epoch 3, Train Loss: 0.32194870710372925, Val Loss: 0.7289478778839111
Epoch 4, Train Loss: 0.2968432903289795, Val Loss: 0.6863986849784851
Epoch 5, Train Loss: 0.2725532650947571, Val Loss: 0.644442617893219
Epoch 6, Train Loss: 0.24903209507465363, Val Loss: 0.6029612421989441
Epoch 7, Train Loss: 0.22625066339969635, Val Loss: 0.5618653893470764
Epoch 8, Train Loss: 0.20420832931995392, Val Loss: 0.5211049914360046
Epoch 9, Train Loss: 0.18293648958206177, Val Loss: 0.48066723346710205
Epoch 10, Train Loss: 0.1624976247549057, Val Loss: 0.44057539105415344
```

GRAPHS :-



